Random Number Generation

Systems Modeling and Simulation
4DV650

Mauro Caporuscio
Department of Computer Science and Media Technology
Mauro.caporuscio@lnu.se

Linnæus University

Why Random numbers

- Random numbers are basic ingredient in simulation
  - Random numbers are used to generate event times and other random variables

- Most computer languages have a function/method/class to generate random numbers:
  - Java: class *java.util.Random*
  - *C++:* function *rand()*
  - *C#:* class *Random*
  - *JS:* function *Math.random()*
  - *Python:* module *random*
  - …

Properties of Random Numbers

- Two important statistical properties:
  - Uniformity
  - Independence
- A random num R_i, must be **independently** drawn from a **uniform distribution** [0,1]

- Uniformity and independence imply that:
  - If the interval [0, 1] is divided into n classes, the expected number of observations in each interval is K/n, where K is the total number of observations
  - The probability of observing a value in a particular interval is independent of the previous values drawn

Generation of Pseudo-Random Numbers

- “**Pseudo**”, because generating numbers using a known method removes the potential for true randomness

- Goal: To produce a sequence of numbers in $[0,1]$ that simulates, or imitates, the ideal properties of random numbers (RN).

- Important considerations in RN methods:
  - Fast
  - Portable to different computers
  - Have sufficiently long cycle
  - Replicable
  - Closely approximate the ideal statistical properties of uniformity and independence

Techniques for Generating Random Numbers

- Linear Congruential Method (LCM).
- Combined Linear Congruential Generators (CLCG).
- Random-Number Streams.

Linear Congruential Method

- To produce a sequence of integers, X_1, X_2, … between *0* and *m-1* by following a recursive relationship:

$$X_{i+1} = (aX_i + c) \bmod m, \quad i = 0,1,2,\ldots$$

The multiplier

The increment

The modulus

- The selection of the values for a, c, m, and X_0 drastically affects the statistical properties and the cycle length.

- The random integers are being generated $[0,m-1]$, and to convert the integers to random numbers:

$$R_i = \frac{X_i}{m}, \quad i = 1,2,\ldots$$

Example

- Use $a = 7^5$, $c = 0$, and $m = 2^{31} - 1$ *(a prime number)*.
- Let $X_0 = 123457$

- The X_i and R_i values are:

  $X_1 = 7^5 * (123,457) \bmod (2^{31} - 1) = 2,074,941,799$ $R_1 = X_1/2^{31} = 0.9662;$

  $X_2 = 7^5 *(2,074,941,799) \bmod (2^{31} - 1) = 559,872,160$ $R_2 = X_2/2^{31} = 0.2607;$

  $X_3 = 7^5 * (559,872,160) \bmod (2^{31} - 1) = 1,645,535,613$ $R_3 = X_3/2^{31} = 0.7662;$

  …

- This method is in actual use and it has been extensively tested.
  - The values for a, c, and m have been selected to ensure that the characteristics desired in a generator are most likely to be achieved.

Note that this method divides by $m + 1$ instead of m; however, for such a large value of m, the effect is negligible

Combined Linear Congruential Generators

- **Reason:** Longer period generator is needed because of the increasing complexity of stimulated systems.
- **Approach:** Combine two or more multiplicative congruential generators.
- Let $X_{i,1}$, $X_{i,2}$, …, $X_{i,j}$,…, $X_{i,k}$, be the i^{th} output from k different congruential generators
  - Assume the j^th generator:
    - Has prime modulus m_j and multiplier a_j and period is m_j-1
    - Produces integers $X_{i,j}$ that are approx uniformly distributed on integers in [1, m-1]
- Then suggested form for the generator is (L'Ecuyer):

$$X_i = \left(\sum_{j=1}^{k} (-1)^{j-1} X_{i,j} \right) \bmod m_1 - 1 \qquad \text{Hence, } R_i = \begin{cases} \dfrac{X_i}{m_1}, & X_i \succ 0 \\ \dfrac{m_1 - 1}{m_1}, & X_i = 0 \end{cases} \qquad P = \frac{(m_1 - 1)(m_2 - 1)\ldots(m_k - 1)}{2^{k-1}}$$

The coefficient: Performs the subtraction

Random-Numbers Streams

- The seed for a linear congruential random-number generator:
  - Is the integer value X_0 that initializes the random-number sequence.
  - Any value in the sequence can be used to “seed” the generator.
- A random-number stream:
  - Refers to a starting seed taken from the sequence X_0, X_1, …, X_P.
  - If the streams are *b* values apart, then stream *i* could defined by starting seed:

$$S_i = X_{b(i-1)}$$

- A single random-number generator with k streams can act like k distinct **virtual** random-number generators

Random Variate Generation

Systems Modeling and Simulation
4DV650

Mauro Caporuscio
Department of Computer Science and Media Technology
Mauro.caporuscio@lnu.se

Linnæus University

Why Random Variate

- In simulation, different **probability distributions** are used to model activities that are generally unpredictable or uncertain
  - e.g., interarrival times, service times, and demands

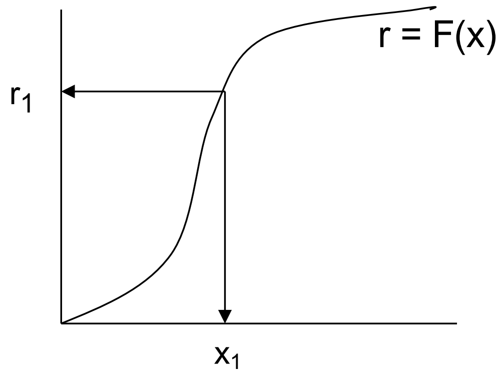
- So far, we have discussed how to generate **uniform** random numbers. How can we generate samples from other distributions?
  - e.g., exponential, uniform, Weibull, triangular, …

- Illustrate some widely-used techniques for generating **random variates**:
  - Inverse-transform technique
  - Acceptance-rejection technique

Inverse-transform Technique

- The concept is to compute the inverse from the cdf:
  - For cdf function: $r = F(x)$
  - Generate r from uniform (0,1)
  - Find x:

$$\boxed{x = F^{-1}(r)}$$



Steps in inverse-transform technique

Step 1. Compute the cdf of the desired random variable X

Step 2. Set F(X) = R on the range of X

Step 3. Solve the equation F(x) = R for X in terms of R

Step 4. Generate uniform random numbers R_1, R_2, R_3, . . . and compute the desired random variates by

$$X_i = F^{-1}(R_i)$$

Some useful random variate

- Exponential distribution with mean λ:

$$X = -\frac{1}{\lambda}\ln(1-R)$$

- Uniform distribution in [a,b]:

$$X = a + (b-a)R$$

- Weibull distribution:

$$X = \alpha[-\ln(1-R)]^{1/\beta}$$

Acceptance-Rejection Technique

- When the inverse cdf does not exist in closed form and therefore the inverse-transform technique is difficult, a viable alternative is the Acceptance-Rejection Technique

- Algorithm:

  Step 1. Generate a random number *R*

  Step 2a. If *R satisfy the condition*, accept $X = R$, then goto Step3.

  Step 2b. If *R does not satisfy* the condition, reject *R*, and goto Step 1.

  Step 3. If another random variate is needed, repeat goto Step 1. If not, stop.

Materials (MyMoodle)

- Reading
  - 03.1 [Lecture] Random Numbers Generation - Reading.pdf
  - 03.2 [Lecture] Random Variate Generation - Reading.pdf
- Exercises (with solutions):
  - 03.1 [Lecture] Random Numbers Generation - Exercises.pdf
  - 03.2 [Lecture] Random Variate Generation - Exercises.pdf

- Play
  - https://www.random.org/integers/